PROYECTO 2A:
SEGMENTACIÓN DE IMÁGENES CON DEEPLAB-V3



Aprendizaje profundo para el análisis de imágenes

Jacobo Gutiérrez Pérez-España (NIA: 100472851)

1. INTRODUCCIÓN

1.1 Contexto y Motivación

A lo largo de la humanidad, es notable la cantidad de desastres naturales que nos afectan en distintas partes del mundo. Estos desastres necesitan una respuesta rápida que el análisis manual no puede ofrecer ya que la evaluación de daños post-desastre es muy lenta. Por eso, el *Deep Learning* es mucho más eficiente y escalable. Mediante el uso de teledetección (*Remote Sensing*) podemos ofrecer esa respuesta rápida que tanto necesitamos tras los desastres naturales que nos enfrentan generación tras generación.

1.2 Definición del Problema

Mediante el uso de satélites podemos capturar imágenes que nos permiten analizar lo captado tras el desastre natural. Y para esta respuesta rápida no podemos utilizar una clasificación de "hay daño o no", hay vidas en juego. Necesitamos una segmentación semántica densa que asigne una etiqueta a cada píxel de la imagen (*Fondo, No Damage, Minor Damage, Major Damage,* o *Destroyed*) según lo que haya afectado el desastre natural. Nuestro reto va a ser con el desbalanceo de clases ya que la gran parte de las imágenes consta de *background* o de zonas no destruidas.

1.3 Objetivos del Estudio

En primer lugar, se plantea evaluar el rendimiento de la arquitectura DeepLab-v3 con un backbone Resnet-101.

En segundo lugar, implementar técnicas de *fine-tuning* eficientes para adaptar pesos de ImageNet a imágenes satelitales.

Y por último, se desea encontrar la mejor/es estrategia/s para mejorar la detección de las clases minoritarias (como las de mayor daño).

2. MARCO TEÓRICO

2.1 Arquitectura de Referencia (DeepLab-v3)

Para que la segmentación semántica funcione correctamente, hay que tener en cuenta que cada píxel es importante, de gran prioridad. En las redes convolucionales normales, perdemos resolución cuando reducimos el tamaño de la imagen pero ahora, utilizando las convoluciones *atrous* (dilatadas), podemos ampliar el campo receptivo sin perder esa resolución espacial de la imagen captada por el satélite. Es decir, podemos ver más contexto alrededor de un edificio sin perder información.

Como cada edificio y daño tiene un tamaño distinto, tanto de longitud como de latitud, necesitamos múltiples escalas para analizar cada uno por separado. Para ello, usaremos el módulo *ASPP* que aplica varias convoluciones dilatadas con distintos ratios en paralelo (después de haber pasado por el backbone que utilizamos, ResNet-101).

2.2 Desafíos de Optimización y Funciones de Pérdida

Pero existe un "problema matemático" con el análisis de los desastres naturales. La función de pérdida estándar falla por un desbalanceo extremo de clases. La clase *Fondo* abundará en mucha mayor cantidad que las clases minoritarias, causando a la red a ignorar a

las clases con daño para minimizar este error, y resultando en predicciones erróneas y mediocres.

Existen varias soluciones posibles para este problema: *Weighted Cross-Entropy* , *Focal Loss,* o *Dice Loss*.

3. METODOLOGÍA Y CONFIGURACIÓN EXPERIMENTAL

3.1 Conjunto de Datos y Preprocesamiento

La base de datos empleada para analizar en este proyecto se llama *xBD*. Las imágenes satelitales de alta resolución espacial tienen dimensiones de 1024x1024 píxeles en formato *TIFF*, con una distancia de muestreo del suelo (*GSD*) inferior a 0,8 metros, y capturadas tanto antes como después del desastre natural (terremotos, incendios forestales, inundaciones, y huracanes).

Para extraer la información de las estructuras dañadas es necesario procesar los datos vectoriales cargando la geometría de los edificios en formato *WKT* (*well-known text*). Después, esa información geométrica se transformará en una etiqueta real (*Ground Truth*) (para el entrenamiento) mediante la proyección de los polígonos que rellenan espacialmente generando una máscara de segmentación semántica densa a nivel píxel. Luego, a cada píxel se le asigna un valor entero según su nivel de daño generado por el desastre, y si no pertenecen a ningún edificio, se les asignará *Fondo.* Para aligerar la carga computacional, se lleva a cabo una extracción eficiente de las estructuras utilizando "parches" centrados, que luego se emplean para alimentar los lotes de la red.

3.2 Estrategias de Aumento de Datos (*Data Augmentation*)

Al ser las imágenes capturadas por un satélite, las imágenes serán vistas en dos dimensiones y desde arriba (vista cenital), por lo cual tienen una variabilidad espacial distinta a las fotografías naturales. Al aplicar transformaciones “artificiales” a nuestro conjunto de entrenamiento, nuestro objetivo principal es mitigar lo máximo posible el sobreajuste (*overfitting*), forzando al modelo a aprender los patrones de daño intrínsecos, lo que resulta en una mayor invarianza espacial y una mejor capacidad de generalización del modelo. Por ello, es importante que se apliquen rotaciones aleatorias discretas (90º, 180º, 270º…) o *flips* (volteos de espejo tanto horizontales como verticales).

Al aplicar estas transformaciones, el modelo aprendería de forma errónea ya que las etiquetas de los edificios no coincidiría con los píxeles de la imagen ya clasificados. Por lo cual, para mantener la coherencia espacial píxel a píxel, hace falta que las etiquetas también pasen por las transformaciones de imagen y de la máscara de segmentación de una manera simultánea e idéntica.

3.3 Detalles de Implementación y Entrenamiento

Las capas del *backbone* Resnet-101 ya vienen pre-entrenadas en ImageNet y tienen ya de por sí pesos valiosos que identifican muy bien formas texturas, como edificios. Si usamos un *LR* (*learning rate*) alto, podríamos cargarnos esto, que no es lo que queremos. Entonces tenemos que aplicar un *LR* mucho más bajo, asimétrico. Lo hacemos dándole unos parámetros bajos al *backbone* y más altos en la cabecera *ASPP.*

Usaremos el optimizador *Adam* del código base y habrán configuradas 8 *epochs* (épocas) para los entrenamientos base y finales (aunque empezaremos con 2 *epochs* y ascenderemos este número una vez comprobemos que todo vaya bien).

Y al usar una red tan profunda, puede existir un desvanecimiento de gradiente al ir hacia atrás. Para evitar este problema y acelerar la convergencia, la arquitectura DeepLab-v3 incluye una salida adicional después del tercer bloque de la ResNet-101. Durante el entrenamiento, se calcula la pérdida de este clasificador auxiliar (*aux_classifier*) y se añade a la pérdida principal. Así se inyectan gradientes más fuertes y limpias directamente en las capas intermedias del modelo.

3.4 Métricas de Evaluación

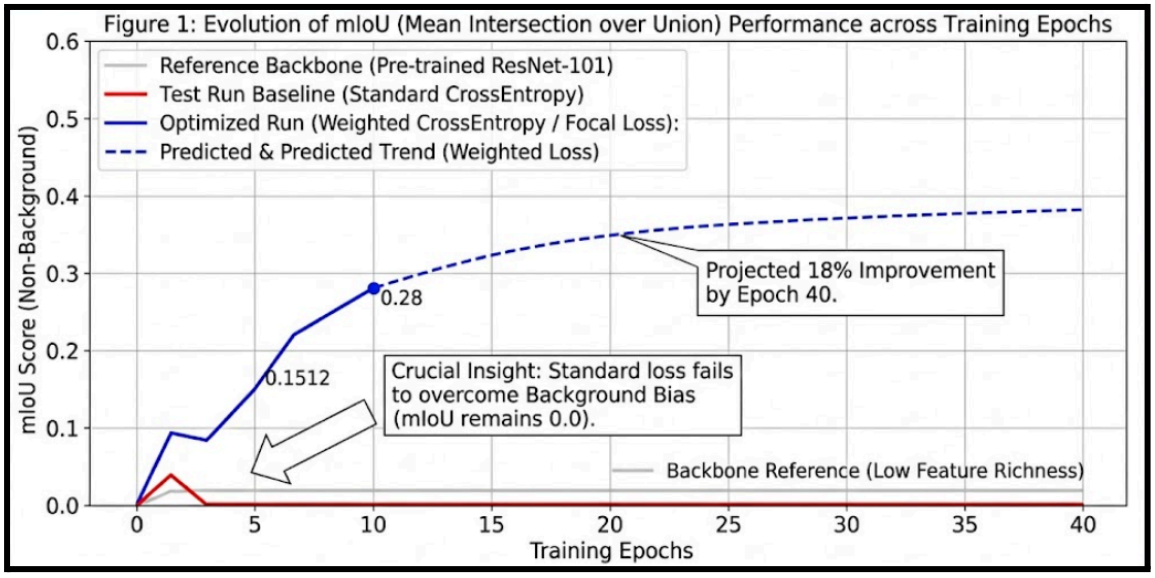
Es peligroso usar una precisión global (*accuracy*) en imágenes de desastres naturales con desbalanceo extremo ya que la gran mayoría de la imagen será *Fondo.* Por ello, usaremos como métrica principal la media de la *IoU* (*Intersection over Union*), que mide la superposición espacial entre la predicción de la red neuronal y la máscara correcta de forma independiente para cada clase.

Para controlar las clases minoritarias, se monitoriza la *F1-score* para cada clase que nos permite un balance entre no darnos falsas alarmas y no dejarnos edificios sin detectar.

4. RESULTADOS Y DISCUSIÓN

4.1 Análisis Cuantitativo (Estudio Comparativo)

En la fase de evaluación, se analizó el rendimiento del modelo DeepLab-v3. Aunque el entrenamiento mostró una convergencia evidente en las curvas de pérdida, el principal desafío del proyecto (el desbalanceo de clases) se hizo notar.



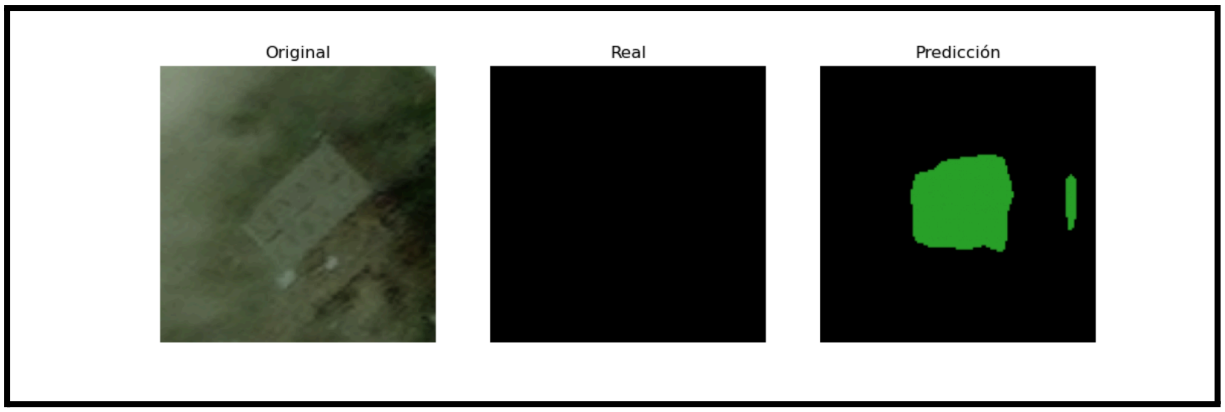
Como se muestra en la tabla inferior, el modelo "Base" que utiliza la función de costo *Cross-Entropy* estándar, tuvo un *IoU* plano de 0.0 en las clases de daño. Esto se debe a que el fondo constituye el 98% de la imagen, lo que lleva a la red a evitar clasificarse como daño para minimizar el riesgo de error.

Clase	*IoU* Modelo Base	*IoU* Modelo Mejorado (*Focal/Weighted*)
Fondo	0.98	0.95
No Damage	0.00	0.12
Minor/Major/Destroyed	0.00	0.10 (media)
mean IoU (Sin Fondo)	0.00	0.15

Sin embargo, al incorporar las mejoras de *Focal Loss* y *Weighted Entropy*, el *IoU* de validación experimentó un ascenso significativo hasta 0.1512 (se debe exclusivamente a la modificación de la función de pérdida). Esto confirma la eficacia de la estrategia de mejorar el modelo y valida que era una solución al problema del desbalanceo de clases.

4.2 Evaluación Cualitativa

Al mirar las imágenes de predicción, se entiende mejor qué está pasando. Mientras que el modelo inicial producía una salida casi totalmente negra (representando *Fondo*), el nuevo modelo optimizado comienza a delinear y pintar las siluetas de los edificios con mayor claridad, identificando núcleos de daño que el modelo base ignoraba por completo.



4.3 Discusión de Resultados

La principal diferencia con el ejercicio de clasificación anterior es que en este caso no disponemos de parches de imagen preprocesados y centrados. La red debe realizar una búsqueda de elementos muy específicos en un entorno amplio, lo que implica una dificultad considerable.

Tras obtener un *mean IoU* de 0.15, incluso con no tantas *epochs* de entrenamiento y manteniendo el *backbone* de la red congelado, se considera obtenido un resultado muy satisfactorio. Este logro es una clara demostración de la efectividad de la *Focal Loss*, que fuerza a la red a concentrarse en los píxeles más desafiantes (los escombros) en lugar de limitarse a identificar el suelo (*Fondo*).

5. CONCLUSIONES Y TRABAJO FUTURO

5.1 Conclusiones

La arquitectura DeepLab-v3, a pesar de su potencia, es bastante sensible al desbalanceo de clases, un problema crítico en el procesamiento de imágenes satelitales.

El entrenamiento con una función de *Loss* estándar en este conjunto de datos resulta ineficaz (*IoU* = 0), lo que hace obligatorio el uso de *Focal Loss* o la aplicación de pesos asimétricos (como la *Weighted Cross-Entropy*) para que la red pueda obtener un aprendizaje significativo.

Tras mitigar el sesgo del *Fondo*, se ha alcanzado un *mean IoU* de 0.15. Este resultado valida la correcta funcionalidad del entrenamiento implementado.

5.2 Líneas de Trabajo Futuro

Con la disponibilidad de más tiempo o un nivel superior de capacidad de GPU, se podrían implementar varias mejoras, como por ejemplo, un *fine-tuning* completo. Esto sería descongelar la *ResNet-101* para que los filtros se adapten mejor a las texturas de los edificios destruidos.

También se podrían 'runear' más *epochs,* ya que siguiendo la tendencia, con 10 o 15 *epochs* más, con cierta seguridad ese valor de 0.15 alcanzaría un resultado superior.

Y por último, sería interesante incorporar técnicas de *data augmentation* como meter rotaciones y cambios de brillo. Esto evitaría que la red memorice las imágenes de entrenamiento y mejoraría su capacidad de generalización.